Diseño e Implementación de una Máquina para Encriptar Mensajes Usando Números Primos Grandes

Santiago Alejandro Céspedes Daza
Facultad de Ingeniería, Universidad Sergio Arboleda
Sistemas Complejos
Bogotá, Colombia

***Resumen*—** Se diseñó e implementó una máquina parametrizable que, dados una técnica, una semilla y un número de cifras, produce números pseudoaleatorios de 40 cifras. La máquina incluye cuatro técnicas: generador congruencial lineal (LCG), Mersenne Twister, SHA-256 en modo contador y Blum-Blum-Shub. Con cada técnica se generaron 10 000 números de 40 cifras y se identificaron los primos mediante Miller-Rabin/Baillie-PSW (verificados con una segunda implementación), obteniendo 87 a 116 primos por técnica frente a 109.3 esperados por el Teorema de los Números Primos. Con los primos hallados se construyó un sistema de encriptación RSA por bloques capaz de cifrar mensajes de longitud arbitraria (UTF-8) y números, y se ejecutó una batería de 25 pruebas de encriptación, superadas en su totalidad. La máquina se puede usar por línea de comandos, mediante un menú interactivo o con una interfaz gráfica. Se documentan además una falla real del LCG con módulo potencia de 2 y las limitaciones de seguridad del sistema.

***Palabras clave*—**Números pseudoaleatorios, PRNG, números primos, RSA, criptografía, Blum-Blum-Shub, Miller-Rabin.

I. LA MÁQUINA

La máquina, implementada en la clase `MaquinaPRNG`, recibe tres parámetros y entrega números de exactamente `cifras` dígitos:

Cuadro 1. Parámetros de entrada de la máquina

Parámetro	Significado
`tecnica`	`lcg`, `mt`, `sha256` o `bbs`
`semilla`	entero; misma semilla ⇒ misma salida
`cifras`	dígitos de la salida (por defecto 40)
`cantidad`	cuántos números emitir (CLI / `lote`)

Cada técnica entrega un entero crudo de $\lceil \log_2(\text{rango}) \rceil + 64$ bits; la máquina lo reduce al intervalo $[10^{c-1}, 10^{c})$ con un módulo. El margen de 64 bits hace que el sesgo de la reducción sea menor que 2^{-64}. Ejemplo de uso:

```
$ python generadores_primos.py maquina \
    --tecnica sha256 --semilla 42 --cifras 40
7968969036072618282685701201480857869219
```

A. Las cuatro técnicas

LCG (Lehmer/Park-Miller [2]): $X_{n+1} = aX_n$ mód $(2^{61} - 1)$, con bloques de 61 bits concatenados. **Mersenne Twister** MT19937: periodo $2^{19937} - 1$, no criptográfico (estado recuperable tras 624 salidas). **SHA-256 en modo contador**: $H = \text{SHA256}(\text{semilla} \,\|\, \text{contador})$; es un CSPRNG. **Blum-Blum-Shub** [3]: $x_{k+1} = x_k^2$ mód pq con $p \equiv q \equiv 3 \pmod 4$ (aquí de 128 bits), emitiendo la paridad de cada x_k; su seguridad se reduce a la dificultad de factorizar pq.

B. Hallazgo: la falla del LCG con módulo 2^{64}

La primera versión del LCG usó los parámetros clásicos de Knuth con módulo 2^{64} y produjo **0 primos en 100 000 números**: con módulo potencia de 2 y a, c impares, el bit menos significativo alterna $0, 1, 0, 1, \ldots$ y todos los números resultan pares [1]. Se corrigió con módulo primo $(2^{61} - 1)$. Hoy el LCG produce 49.8 % de pares, como se espera.

II. EXPERIMENTO: 10 000 NÚMEROS DE 40 CIFRAS POR TÉCNICA

Con semilla 2026 se generaron 10 000 números por técnica (40 000 en total). Se verificó que todos tuvieran exactamente 40 cifras y que no hubiera repetidos (0 en las cuatro técnicas). La primalidad se evaluó con `gmpy2.is_prime` (Miller-Rabin + Baillie-PSW). Como control independiente, *todos* los primos hallados y 600 compuestos por técnica se revisaron con `sympy.isprime`: coincidieron en el 100 % de los casos.

Por el Teorema de los Números Primos la densidad cerca de x es $\approx 1/\ln x$; en el centro del rango ($x \approx 5 \times 10^{39}$) vale 1.0928 %, es decir **109.3 primos esperados** en 10 000 números, con desviación típica binomial $\sigma \approx 10{,}4$.

Cuadro 2. Primos hallados en 10 000 números de 40 cifras (semilla 2026)

Técnica	Primos	Densidad	z
LCG $(2^{61} - 1)$	106	1.060 %	$-0{,}32$
Mersenne Twister	116	1.160 %	$+0{,}65$
Hash SHA-256	87	0.870 %	$-2{,}14$
Blum-Blum-Shub	114	1.140 %	$+0{,}45$
Esperado (TNP)	109.3	1.093 %	—

Tres técnicas caen a menos de $0{,}7\sigma$ de lo esperado. SHA-256

queda a $-2{,}14\sigma$; con cuatro técnicas, un valor así aparece por azar con probabilidad cercana al 12 %, y para comprobar que no es un defecto del generador se repitió el experimento con las semillas 1 a 10: el promedio de primos fue 110.1 (LCG), 104.8 (MT), 109.4 (SHA-256) y 107.6 (BBS), todos compatibles con 109.3 (error típico del promedio $\approx 3{,}3$). Los primos hallados (423 en total) se guardan en `primos_encontrados.csv`. Ejemplos:

```
LCG   693304141368651403456574244046747215667 7
MT    529472536301641019034072267821768313808 3
SHA256 971954484777050200005938321124948422495 9
BBS   777460259028434665395921001094702211864 7
```

BBS es la técnica más lenta (1.10 s para 10 000 números, frente a $\leq 0{,}01$ s del resto) porque emite un bit por elevación al cuadrado modular; el test de primalidad de los 10 000 números toma $\approx 0{,}01$ s por técnica.

III. SISTEMA DE ENCRIPTACIÓN CON LOS PRIMOS ENCONTRADOS

Se construyó un RSA [4] cuyos factores son dos de los primos de 40 cifras hallados: $n = pq$, $\varphi = (p-1)(q-1)$, $e = 65537$, $d = e^{-1} \bmod \varphi$, con $c = m^e \bmod n$ y $m = c^d \bmod n$. Se exige que $p \neq q$, que ambos sean primos y que $\gcd(e, \varphi) = 1$.

A. *Cifrado por bloques*

Con n de 266 bits un mensaje cabe en un solo entero apenas si tiene 32 bytes, así que el sistema trocea el texto UTF-8 en bloques de 32 bytes. A cada bloque se le antepone el byte `0x01`, que conserva los ceros iniciales y garantiza $m < n$. Cada bloque se cifra por separado; el mensaje cifrado es la lista de enteros resultantes. También se pueden cifrar directamente números $0 \leq m < n$.

```
p (LCG) = 693304141368651403456574244046747215667 7
q (SHA256) = 971954484777050200005938321124948422495 9
n = p*q (80 cifras, 266 bits), e = 65537
Mensaje : 'HOLA RSA DESDE SISTEMAS COMPLEJOS'
Cifrado[0]: 42935951828314565587...460
Cifrado[1]: 39423169855720735276...989
Descifrado: igual al original -> True
```

IV. PRUEBAS DE ENCRIPTACIÓN

Se ejecutó una batería de 25 pruebas automáticas (Tabla 3); todas se superaron. Las pruebas P13 a P16 validan la máquina y los primos; P17 a P19 amplían las de cifrado, y P19 muestra el ataque de factorización contra un RSA pequeño. Los 30 sistemas RSA de la prueba P1 se formaron con los primeros 5 primos de cada una de las 6 combinaciones de técnicas distintas.

Cuadro 3. Resultados de las pruebas de encriptación

Id	Prueba	Res.
P1	$ed \equiv 1 \pmod{\varphi}$ en 30 pares (técnicas distintas)	30/30
P2	Ida y vuelta: texto corto, vacío, frase, Unicode (ñ, kanji, emoji), 2000 bytes, ceros iniciales	6/6
P3	Mismo mensaje con las 30 llaves	30/30
P4	204 números (0, 1, 2, $n-1$ y 200 aleatorios) y 10 primos de 40 cifras	214/214
P5	El cifrado no coincide con el texto plano	OK
P6	Llave privada incorrecta no recupera el mensaje	OK
P7	Alterar 1 bit del cifrado destruye el mensaje	OK
P8	Avalancha: 1 carácter cambia 139/265 bits (52.5 %)	OK
P9	Errores controlados: $m \geq n$, $m < 0$, $p = q$, p compuesto	4/4
P10	$E(m_1)E(m_2) = E(m_1 m_2)$ (homomorfismo)	OK
P11	Determinismo: mismo texto, mismo cifrado	OK
P12	100 kB: cifrar 0.03 s, descifrar 0.44 s (3125 bloques)	OK
P13	Reproducibilidad: misma semilla, misma salida; otra semilla, otra salida	OK
P14	La máquina respeta 10, 25, 40 y 60 cifras (4 técnicas $\times$ 100 números)	OK
P15	Dígitos uniformes: χ^2 máximo 18.25 $<$ 27,88 ($p = 0{,}001$, 9 g.l.)	OK
P16	Los 423 primos hallados pasan `sympy.isprime`	OK
P17	Ninguna de las otras 29 llaves descifra el mensaje	OK
P18	Intercambiar bloques cifrados altera el mensaje	OK
P19	Un n de $\sim$20 cifras se factoriza y se obtiene d	OK

Las pruebas P10 y P11 *confirman* dos debilidades conocidas del RSA de libro: es determinista (un atacante puede comprobar adivinanzas de texto) y es maleable (se pueden fabricar cifrados válidos de productos de mensajes). Los sistemas reales usan relleno aleatorio (OAEP) para evitarlo; aquí no se implementó.

A. *Limitaciones de seguridad*

Con n de 266 bits (80 cifras) el sistema es **didáctico, no seguro**: un número de ese tamaño se factoriza con criba del campo numérico en tiempo corto con un computador común, tras lo cual se obtiene d. Un RSA actual exige $n \geq 2048$ bits. El mismo programa serviría con primos de 300 cifras, pero no con `mt` ni `lcg` como fuente, ya que ambos son predecibles; para uso real se debe usar un CSPRNG (SHA-256, BBS con parámetros grandes o el generador del sistema operativo). El caso de OpenSSL en Debian [5], donde un generador debilitado dejó solo $\sim$32 768 claves posibles, ilustra el riesgo.

V. INTERFACES DE USO

El programa se ejecuta con `python generadores_primos.py`, que abre directamente un **menú interactivo** en consola con ocho opciones: generar números con la máquina, ejecutar el experimento de 10 000 números, cifrar un mensaje, descifrar bloques pegados por el usuario,

correr las 25 pruebas, ejecutar la demo completa, **abrir la interfaz gráfica** y salir. La interfaz gráfica (`tkinter`) tiene cuatro pestañas: máquina PRNG, experimento (tabla de primos por técnica), cifrado/descifrado de mensajes y pruebas. Cada función también puede invocarse sin menú (`maquina`, `experimento`, `cifrar`, `pruebas`, `todo`, `gui`). Se probó cada opción del menú y se accionaron automáticamente los botones de la interfaz gráfica, que cifró y descifró correctamente un mensaje con Unicode y emoji y reportó 25/25 pruebas superadas.

VI. CONCLUSIONES

Se construyó una máquina parametrizable con cuatro técnicas independientes que produce números pseudoaleatorios de 40 cifras; con ella se generaron 10 000 números por técnica y se hallaron 87 a 116 primos, consistentes con los 109.3 predichos por el Teorema de los Números Primos (confirmado con 10 semillas adicionales y con una segunda implementación de la primalidad). Con los 423 primos obtenidos se construyó un sistema RSA por bloques que superó las 25 pruebas de encriptación, incluyendo texto Unicode, mensajes largos, números y casos de fallo. El sistema demuestra el algoritmo de principio a fin, pero por el tamaño de sus llaves no protege datos reales.

REFERENCIAS

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3.ª ed. Addison-Wesley, 1997.

[2] S. K. Park y K. W. Miller, “Random number generators: good ones are hard to find,” *Commun. ACM*, vol. 31, no. 10, pp. 1192–1201, 1988.

[3] L. Blum, M. Blum, y M. Shub, “A simple unpredictable pseudo-random number generator,” *SIAM J. Comput.*, vol. 15, no. 2, pp. 364–383, 1986.

[4] R. L. Rivest, A. Shamir, y L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Commun. ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[5] L. Weinmann, “Debian OpenSSL predictable random number generator,” CVE-2008-0166, 2008.